CMSC 27200 - Spring 2025 — Lorenzo Orecchia
due 4/11/24 via Gradescope

H3.

- The assignment is due at Gradescope on 4/11/24.
- A LaTeX template will be provided for each homework. You are strongly encouraged to type your homework into this template using LaTeX. If you are writing by hand, please fill in the solutions in this template, inserting additional sheets as necessary. This will help facilitate the grading.
- You are permitted to discuss the problems with up to 2 other students in the class (per problem); however, *you must write up your own solutions, in your own words*. Do not submit anything you cannot explain. If you do collaborate with any of the other students on any problem, please list all your collaborators in the appropriate spaces.
- Similarly, please list any other source you have used for each problem, including other textbooks or websites.
- *Show your work.* Answers without justification will be given little credit.
- Your homework is *resubmittable*. Please refer to the course syllabus on Canvas for a more detailed description of this. For any problem that you have not changed from your last submission, please make sure to indicate this in your submission to help our graders grade faster.
- Each student can request a total of up to 5 days of extensions on all homework - either for original submission or resubmission. Up to two days can be requested for a homework. If you would like to request an extension, please fill out the Homework extension form. Extensions do not carry any late penalty. If there is a life emergency that might prevent you from submitting on time not covered by this policy, email the Head TA.
- Taking into account the different focus students have coming into this course, you can choose between a more advanced theory problem or a programming assignment. **Complete either theory problem 4 or programming problem 5.**

Problem 1 (Recursive functions) *For the following recursive functions, compute their $O(\cdot)$ complexity:*

1. $T(n) = 7T(n/2) + 4n$
2. $T(n) = 16T(n/4) + n^2$
3. $T(n) = 2T(n/4) + \sqrt{n}$
4. $T(n) = T(n-1) + n^2$

Collaborators:

Solution: Your solution here.

Extra space for your solution

Problem 2 (Mafia Game) *A group of $n > 2$ friends are playing a game of Mafia. In each round of this game, the n players can select any subset of the n players to interrogate. Some number of the players are **Mafia**, and they will be revealed during the interrogation.*

Denote by $P = \{P_1, P_2, \ldots P_n\}$ the set of n players in the game. We may represent the interrogation as a function I that takes as input any non-empty subset of P and outputs a boolean in $\{\textsc{True}, \textsc{False}\}$ indicating whether the subset contains a Mafia member. More formally, if P_i is Mafia, then $P_i \in P' \implies I(P') = \textsc{True}$.

*Suppose that there is only one Mafia member among the n. We will design a divide-and-conquer procedure that allows players to identify the mafia member in $O(\log n)$ interrogations. Remember that every divide-and-conquer algorithm can be split into three parts: **divide**, **conquer**, and **process**.*

*(a) Provide a description (in words) of the **divide** step of the procedure. (i.e. How will you split up the input to process it?)*

*(b) Provide a description (in words) of the **conquer** step of the procedure. (i.e. How will you "solve" each section of the input?)*

*(c) Provide a description (in words) of the **process** step of the procedure. (i.e. How will you use your answers from the **conquer** step?)*

(d) Provide a proof sketch that this procedure always finds the Mafia.

(e) Provide a proof sketch that your procedure from part (a) only takes $O(\log n)$ tests.

(f) Provide a formal proof that no procedure can guarantee finding the Mafia member in less than $\lfloor \log_2 n \rfloor$ interrogations.

(g) (Bonus - Just to think about, no need to submit anything.) Suppose the Mafia member knows your procedure for picking subsets, and can successfully lie in k interrogations (for some constant k). Obviously, if we just run each interrogation from (a) $k+1$ times, we'll find the Mafia. That seems redundant though. Is there a way for us to guarantee that we'll identify the Mafia in less than $(k+1)\log n$ interrogations?

(Not bonus - Submit this) Suppose instead that there are two Mafia in the group of n players. Furthermore, they can cover each other's tracks in the interrogation: if both are in the same interrogation, then the players won't be able to detect that they are Mafia!

Formally, there exist two players P_i and P_j with $i \neq j$ such that

$$I(P') = \begin{cases} \textsc{False} & \text{If neither } P_i \text{ nor } P_j \text{ are in } P', \\ \textsc{True} & \text{If exactly one of } P_i \text{ or } P_j \text{ are in } P', \\ \textsc{False} & \text{If both } P_i \text{ and } P_j \text{ are in } P'. \end{cases}$$

(h) Describe a procedure for selecting subsets to interrogate that allows the players to identify both Mafia in $O(\log n)$ interrogations. You may invoke your algorithm from part (a).

(i) Sketch a proof that your procedure for part (h) is correct.

(j) Sketch a proof that your procedure from part (h) takes $O(\log n)$ interrogations.

Collaborators:

Solution: Your solution here.

Extra space for your solution

PROBLEM 3 (TILINGS) *Consider a square $N \times N$ grid where $N = 2^k$ is a power of 2, and imagine placing L-shaped domino pieces of area 3 on this grid. A **tiling** of the grid is a way of placing the pieces so that no two pieces overlap and every single grid cell is covered. In this problem you will be tasked with describing a procedure (an algorithm!) to cover the grid using only these L-shaped pieces. In the input grid, one square is excluded, and doesn't need to be covered by any domino piece.*

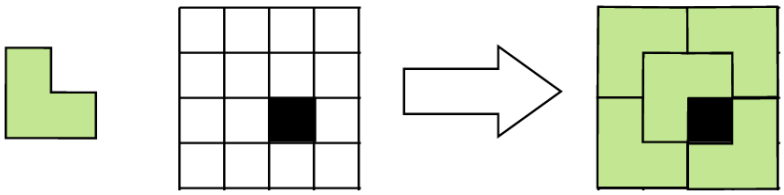


Figure 1: You are given L-shaped domino pieces (in light green) and a grid, and you are asked to find a way to tile the entire grid (excluding the black square). An example of a valid tiling is shown on the right.

(a) *Verify that, if $N = 2^k$, then $N^2 \equiv 1 \bmod 3$, and explain why this condition is necessary (while a priori not sufficient) for a valid tiling to exist.*

(b) *Describe a procedure that allows you to find a valid tiling, given any input grid as above. You may give pseudocode but you are not required to.*

(c) *Sketch a proof that your procedure works correctly.*

(d) *Reflect on your problem-solving process. What was one key insight that allowed you to design this procedure? (Just a sentence or two is fine, your answer will be accepted as long as it is reasonable.)*

Note: every domino piece has the shape described above and covers exactly three squares, every input grid has exactly one excluded (black) square, but the location of the black square may change between input grids.

Collaborators:

Solution: Write your solution here.

Extra space for your solution

You can choose to complete this theory problem **or** the programming assignment

PROBLEM 4 (MAXIMUM INTERVAL OVERLAP) *An **interval** on the one-dimensional real number line can be defined as a pair of starting and terminating one-dimensional coordinates.*

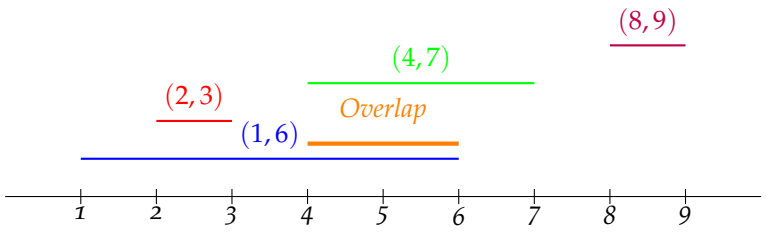
***Example**. The interval $(-4, 6)$ represents the set of real numbers between -4 and 6.*

*The **overlap** between two intervals (s_1, t_1) and (s_2, t_2) is defined as the interval $(\max(s_1, s_2), \min(t_1, t_2))$, and the **size** of an interval (s, t) is defined as $\max(t - s, 0)$ (i.e., if $t < s$, the size of the interval is 0).*

***Example**. The overlap between (-4, 6) and (1, 8) is the interval (1, 6), with size 5.*

*In this problem, our goal is to determine the **size of the largest overlap** between any pair of intervals, given some collection of intervals.*

***Example**. Consider a set of 4 intervals: $(1, 6), (2, 3), (4, 7), (8, 9)$. The largest overlap is between $(1, 6)$ and $(4, 7)$: their overlap is $(4, 6)$ which has size 2.*



(a) *Write **pseudocode** for a **recursive divide-and-conquer** algorithm which takes an array $A[1 \ldots n]$, where each $A[i] = (s_i, t_i)$ represents an interval, and returns the **size of the largest overlap** between any pair of intervals in A. The intervals in A are sorted by the starting coordinate s_i. Your algorithm should run in $O(n \log n)$ time.*

(b) ***Prove** your algorithm is correct. Use induction.*

(c) *Write and solve a running time recurrence for your divide-and-conquer algorithm in part (a) in terms of n.*

(d) *Write **pseudocode** for a generalized algorithm that finds the k largest overlaps between distinct pairs of intervals. Your algorithm should now take an additional parameter k which can be any number between 1 and $\binom{n}{2}$. Your algorithm from part (a) solves the case where $k = 1$.*

(e) *Write and solve a running time recurrence for your divide-and-conquer algorithm in part (d) in terms of n and k.*

(f) *What is the space complexity of the working memory used by your algorithm in part (d), excluding the memory needed to store the inputs? Give your answer in asymptotic notation.*

Collaborators:

***Solution:** Write your solution here.*

Extra space for your solution

You can choose to complete this programming problem **or** the previous theory assignment

Problem 5 (Drink of Choice)

The world's largest Starbucks is in downtown Chicago. Starbucks Reserve Roastery occupies five stories and 35,000 square feet. Such a large location has an equally large number of baristas, and of course, a very long line of customers. As you wait for your drink of choice, you notice that while each barista might take a different amount of time to prepare a drink, they are perfectly consistent **across** drinks.

The fifth barista might always take 3 minutes to make a drink, but the second barista always takes 7 minutes instead. If multiple baristas are available at the same time, the next customer goes to the barista with the **lower number**. As you wait in line, you decide to find out which barista is going to prepare your drink and when it will be ready.

Assignment

Given a list of N integers t_i, representing the time it takes for each barista to prepare a drink, find which barista is going to serve you and when you are getting your drink.

You should implement your algorithm in `barista.py` by completing the `solve()` function. This function takes an integer N representing the number of baristas, an integer K representing your position in line, and a list of integers `times` where the i-th element is the time it takes barista i to prepare a drink. The baristas are numbered 1 to N, and the places in line also start with 1.

Your code must return two integers, the number of the barista that is going to serve you and when your drink is going to be ready.

You may not use any external libraries beyond those in `requirements.txt` and Python's built-in modules.

Download the necessary code from Github here.

Testing

A number of test cases have been provided, and you can test your implementation by using the `pytest` library.

Install the required libraries by running the following command:

```
python3 -m pip install -r requirements.txt
```

Then you can run all the tests at once with the following command:

```
python3 -m pytest test_barista.py
```

The test cases and expected outputs are located in the `testcases` folder. If you want, you may add additional test cases by adding to this folder (but please do not modify the test cases we've provided).

On the first line, there are two integers, N and K, representing the number of baristas and your place in line. The next line contains N integers, the time it takes barista i to prepare a drink.

The output files contain two integers in a single line, the number of the barista that will serve you and the time it will take for your drink to be ready.

Note that you do not need to do any parsing of these files yourself — that is handled for you by our tests.

You can invoke `pytest` in a number of ways. If you'd like, you can read up on that here. Most notably, you can run a single test on its own with the `-k` flag. For instance:

```
python3 -m pytest -k input05
```

This command runs the test from `input05.txt`.

It can also be nice to see only the first test that fails, which can be done with the `-x` flag. Finally, the `-v` flag will make test output more verbose, which can help you diagnose if tests are failing.

Example

Here are some example inputs and outputs. You are given the times, and you need to output the correct barista and time.

Sample Input	Sample Output
2 4 10 5	1 20
3 3 1 1 1	3 1
5 4 1 1 1 2 3 4 5	1 180
5 4 1 4 1 2 3 4 5	3 183

Limits and Notes

$1 \leq N \leq 10^5$
$1 \leq t_i \leq 10^5$
$1 \leq K \leq 10^9$

The time your code has to pass any one test is **2 seconds**. The tests will timeout if your code takes longer than this time. Different machines run at different speeds, so the machine that matters here is the one that Gradescope uses when it tests your code after a submission (this is the output we will read when we grade your submission).

Note that 10^9 is quite large, so any algorithm that is $O(K)$ runtime will take too long. Likewise, algorithms that take $O(\max(t_i) \cdot N)$ time will also take too long. Keep these facts in mind as you develop your algorithms.

It is also tempting to consider skipping time intervals of size $\text{lcm}(t_1, t_2, ..., t_N)$ and searching linearly on the final interval, but this doesn't quite work because this value can be very large (consider if all t_i are prime, for example).

Submission

Submit your code by uploading your `barista.py` to the separate coding assignment on Gradescope. This assignment is due at the same time as Homework 3 theory problems, and the same late policy applies.